Datasets and Named Graphs

Simon Frost

Overview

While an RDF Graph is a set of triples, real-world applications often need multiple graphs with different provenance. An RDF **Dataset** holds a default graph plus named graphs, each identified by a URI.

```
using RDFLib
ex = Namespace("http://example.org/")
```

```
Namespace("http://example.org/")
```

Creating a Dataset

```
ds = Dataset()

# Add triples to named graphs
add!(ds, Triple(ex("alice"), RDF.type, FOAF("Person")), ex("graph/alice"))
add!(ds, Triple(ex("alice"), FOAF("name"), Literal("Alice")), ex("graph/alice"))
add!(ds, Triple(ex("alice"), FOAF("mbox"), URIRef("mailto:alice@example.org")), ex("graph/ali

add!(ds, Triple(ex("bob"), RDF.type, FOAF("Person")), ex("graph/bob"))
add!(ds, Triple(ex("bob"), FOAF("name"), Literal("Bob")), ex("graph/bob"))

# Add to the default graph
add!(ds, Triple(ex("graph/alice"), DCTERMS("creator"), Literal("System A")))
add!(ds, Triple(ex("graph/bob"), DCTERMS("creator"), Literal("System B")))

println("Dataset has $(length(collect(graphs(ds)))) graphs (including default)")
```

```
Dataset has 3 graphs (including default)
```

Accessing Named Graphs

```
# Retrieve a specific named graph
alice_g = get_graph(ds, ex("graph/alice"))
println("Alice's graph: $(length(alice_g)) triples")
for t in alice_g
    println("  ", t)
end
```

```
Alice's graph: 3 triples
  (URIRef("http://example.org/alice"), URIRef("http://www.w3.org/1999/02/22-
rdf-syntax-ns#type"), URIRef("http://xmlns.com/foaf/0.1/Person"))
  (URIRef("http://example.org/alice"), URIRef("http://xmlns.com/foaf/0.1/name"), Literal("Ali
  (URIRef("http://example.org/alice"), URIRef("http://xmlns.com/foaf/0.1/mbox"), URIRef("mail
```

Quads

In a dataset, triples carry a fourth component — the graph name (context):

```
println("All quads:")
for q in quads(ds)
    ctx = q.graph === nothing ? "default" : string(q.graph)
    println("  [$(ctx)] $(q.subject) $(q.predicate) $(q.object)")
end
```

```
All quads:
  [default] URIRef("http://example.org/graph/alice") URIRef("http://purl.org/dc/terms/creator
  [default] URIRef("http://example.org/graph/bob") URIRef("http://purl.org/dc/terms/creator")
  [http://example.org/graph/alice] URIRef("http://example.org/alice") URIRef("http://www.w3.o
rdf-syntax-ns#type") URIRef("http://xmlns.com/foaf/0.1/Person")
  [http://example.org/graph/alice] URIRef("http://example.org/alice") URIRef("http://xmlns.co
  [http://example.org/graph/alice] URIRef("http://example.org/alice") URIRef("http://xmlns.co
  [http://example.org/graph/bob] URIRef("http://example.org/bob") URIRef("http://www.w3.org/1
rdf-syntax-ns#type") URIRef("http://xmlns.com/foaf/0.1/Person")
  [http://example.org/graph/bob] URIRef("http://example.org/bob") URIRef("http://xmlns.com/fo
```

Serializing Datasets

TriG Format

TriG extends Turtle with named graph blocks:

```
trig = serialize_trig(ds)
println(trig)
```

```
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

{
    ns1:alice ns2:creator "System A" .

    ns1:bob ns2:creator "System B" .
}

ns1:alice {
    ns3:alice a ns4:Person ;
        ns4:mbox <mailto:alice@example.org> ;
        ns4:name "Alice" .
}

ns1:bob {
    ns3:bob a ns4:Person ;
        ns4:name "Bob" .
}
```

N-Quads Format

One quad per line:

```
nq = serialize_nquads(ds)
println(nq)
```

```
<http://example.org/graph/alice> <http://purl.org/dc/terms/creator> "System A" .
<http://example.org/graph/bob> <http://purl.org/dc/terms/creator> "System B" .
```

```
<http://example.org/alice> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://xmlns.co
<http://example.org/alice> <http://xmlns.com/foaf/0.1/name> "Alice" <http://example.org/graph
<http://example.org/alice> <http://xmlns.com/foaf/0.1/mbox> <mailto:alice@example.org> <http
<http://example.org/bob> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://xmlns.com/
<http://example.org/bob> <http://xmlns.com/foaf/0.1/name> "Bob" <http://example.org/graph/bob
```

Conjunctive Graphs

A `ConjunctiveGraph` provides a unified view over all graphs in a dataset:

```
cg = ConjunctiveGraph()

# Add triples to different contexts
ctx1 = ex("context/weather")
ctx2 = ex("context/geography")

add!(cg, Triple(ex("london"), ex("temp"), Literal(15)), ctx1)
add!(cg, Triple(ex("paris"), ex("temp"), Literal(18)), ctx1)
add!(cg, Triple(ex("london"), RDF.type, ex("City")), ctx2)
add!(cg, Triple(ex("paris"), RDF.type, ex("City")), ctx2)

# Query across all contexts
println("All triples across contexts:")
for t in cg
    println("  ", t)
end
```

```
All triples across contexts:
  (URIRef("http://example.org/london"), URIRef("http://www.w3.org/1999/02/22-
rdf-syntax-ns#type"), URIRef("http://example.org/City"))
  (URIRef("http://example.org/paris"), URIRef("http://www.w3.org/1999/02/22-
rdf-syntax-ns#type"), URIRef("http://example.org/City"))
  (URIRef("http://example.org/london"), URIRef("http://example.org/temp"), Literal("15", data
  (URIRef("http://example.org/paris"), URIRef("http://example.org/temp"), Literal("18", datat
```

```
# Get a specific context
weather = get_context(cg, ctx1)
println("\nWeather context:")
for t in weather
    println("  ", t)
end
```

```
Weather context:
  (URIRef("http://example.org/london"), URIRef("http://example.org/temp"), Literal("15", data
  (URIRef("http://example.org/paris"), URIRef("http://example.org/temp"), Literal("18", datat
```

Read-Only Aggregate

Combine multiple graphs into a single read-only view:

```
g1 = parse_rdf("""
    @prefix ex: <http://example.org/> .
    ex:a ex:p "1" .
""", TurtleFormat())

g2 = parse_rdf("""
    @prefix ex: <http://example.org/> .
    ex:b ex:q "2" .
""", TurtleFormat())

agg = ReadOnlyGraphAggregate([g1, g2])
println("Aggregate has $(length(agg)) triples:")
for t in agg
    println("  ", t)
end
```

```
Aggregate has 2 triples:
  (URIRef("http://example.org/a"), URIRef("http://example.org/p"), Literal("1"))
  (URIRef("http://example.org/b"), URIRef("http://example.org/q"), Literal("2"))
```

Use Case: Provenance Tracking

Named graphs are commonly used to track where data came from:

```
ds2 = Dataset()
prov = Namespace("http://www.w3.org/ns/prov#")

# Data from a sensor
sensor_g = add_graph(ds2, ex("graph/sensor"))
add!(sensor_g, Triple(ex("reading1"), ex("temperature"), Literal(22.5)))
add!(sensor_g, Triple(ex("reading1"), ex("humidity"), Literal(65)))
```

```
# Record provenance in the default graph
add!(ds2.default_graph, Triple(ex("graph/sensor"), RDF.type, prov("Entity")))
add!(ds2.default_graph, Triple(ex("graph/sensor"), prov("wasGeneratedBy"), ex("sensorProcess
add!(ds2.default_graph, Triple(ex("graph/sensor"), prov("generatedAtTime"),
    Literal("2025-01-15T10:30:00"; datatype=XSD.dateTime)))

println("Dataset with provenance:")
for t in ds2.default_graph
    println("  ", t)
end
```

```
Dataset with provenance:
  (URIRef("http://example.org/graph/sensor"), URIRef("http://www.w3.org/1999/02/22-
rdf-syntax-ns#type"), URIRef("http://www.w3.org/ns/prov#Entity"))
  (URIRef("http://example.org/graph/sensor"), URIRef("http://www.w3.org/ns/prov#wasGeneratedB
  (URIRef("http://example.org/graph/sensor"), URIRef("http://www.w3.org/ns/prov#generatedAtTi
01-15T10:30:00", datatype=URIRef("http://www.w3.org/2001/XMLSchema#dateTime")))
```